Container Orchestration & Resource Isolation System

Project Documentation for Academic Implementation

Executive Summary

This document outlines a comprehensive academic project to build a lightweight container orchestration and resource isolation system from scratch. The project demonstrates core operating system concepts including process management, memory isolation, resource scheduling, and inter-process communication through practical implementation[1][2].

The system provides containerization capabilities similar to Docker and Kubernetes, implementing isolated process environments with controlled resource allocation. This hands-on approach enables deep understanding of kernel-level operations, namespace implementation, and distributed system design[3][6].

1. Introduction

1.1 Project Overview

Container orchestration automates the deployment, management, scaling, and lifecycle of containerized applications in complex environments[1]. This project builds a simplified orchestration system that creates isolated process environments with controlled CPU, memory, and storage allocation.

1.2 Learning Objectives

- Understand process creation, termination, and lifecycle management
- Implement memory isolation using virtual memory techniques
- Design file system virtualization and storage quota enforcement
- Build fair CPU scheduling algorithms with resource limits
- Develop inter-container communication mechanisms
- Prevent deadlocks in multi-container scenarios
- Create namespace-based isolation at the kernel level

1.3 Technology Foundation

Linux containers utilize kernel features including namespaces, cgroups, and resource limits to achieve isolation[6]. Namespaces create separate views of system resources (PID, mount, network, IPC, UTS, user), while cgroups enforce resource allocation boundaries[7][10].

2. Core Operating System Concepts

2.1 Process Creation & Termination

Process Lifecycle Management

Containers represent isolated process trees within the operating system. Each container maintains its own process hierarchy with PID namespace isolation[6].

Key Implementation Aspects:

- Fork and exec system calls for process creation
- Parent-child process relationships
- Process state transitions (running, waiting, terminated)
- Zombie process prevention and cleanup
- Graceful shutdown and forced termination

Namespace Hierarchy:

Parent namespaces can observe all processes in child namespaces, but children remain isolated from each other[6]. This asymmetric

visibility enables system monitoring while maintaining workload isolation.

2.2 Memory Isolation & Virtual Memory

Virtual Memory for Containers

Each container operates within its own virtual address space, isolated from other containers[9]. The system implements memory partitioning with enforced boundaries.

Implementation Components:

- Virtual address space allocation per container
- Page table management for memory translation
- Memory limits enforcement (hard and soft limits)
- Memory reservation and overcommitment policies
- Out-of-memory (OOM) handling and container termination

Memory Resource Configuration:

Memory limits prevent individual containers from monopolizing system resources[7]. The system enforces both reservation (guaranteed memory) and maximum limits (ceiling).

2.3 File System Virtualization

Isolated File Systems

Container orchestration provides each container with an isolated file system view using mount namespaces[6][12].

Core Features:

- chroot and pivot_root for root directory isolation
- Read-only remounting for immutable base images
- Custom /proc and tmpfs mount points
- Overlay file systems for layered storage
- Storage quota enforcement per container

Directory Isolation:

Containers cannot access files outside their designated directory tree, preventing unauthorized access and conflicts[12].

2.4 Resource Allocation & Limits

CPU Time Allocation

The scheduler distributes CPU time across containers using weighted allocation based on CPU shares[10].

Resource Configuration Model:

Resource Type	Configuration Method	Enforcement Mechanism
CPU	CPU shares (weighted) or CPU quota (hard limit)	Linux cgroups with proportional scheduling
Memory	Reservation + maximum limit	Memory cgroups with OOM killer
Storage	Disk quota per container	File system quota or overlay limits
Network	Bandwidth limits	Traffic control (tc) rules

Table 1: Resource allocation strategies for container isolation

Fair Scheduling:

CPU shares act as weights determining relative CPU access during contention[10]. A container with 2048 shares receives twice the CPU time of a container with 1024 shares when both compete for resources.

2.5 Process Synchronization

Inter-Container Communication (IPC)

While containers are isolated, controlled communication channels enable coordination[9].

Synchronization Primitives:

- Shared memory segments with access control
- Semaphores for mutual exclusion
- Message queues for asynchronous communication
- Unix domain sockets for stream communication
- Network sockets for distributed communication

IPC Namespace Isolation:

The IPC namespace provides each container with its own set of System V IPC objects and POSIX message queues[9], preventing unintended interference.

2.6 Deadlock Prevention

Multi-Container Deadlock Scenarios

Deadlocks occur when containers hold resources while waiting for others, creating circular dependencies.

Prevention Strategies:

- Resource ordering (acquire resources in consistent order)
- Timeout-based resource acquisition
- Deadlock detection and recovery algorithms
- Resource graphs for circular dependency detection
- Priority-based resource allocation

Banker's Algorithm:

Implement safe state verification before resource allocation to prevent deadlock in multi-container environments.

2.7 Namespace Implementation

Linux Namespace Types

Namespaces provide the foundation for container isolation[6][12]:

Namespace Type	Isolation Provided
PID	Process IDs and process tree
Mount	File system mount points
Network	Network interfaces, routing tables, firewall rules
IPC	Inter-process communication resources
UTS	Hostname and domain name
User	User and group IDs, privilege isolation
Cgroup	Control group hierarchy

Table 2: Linux namespace types and their isolation scope

Namespace Creation:

Use the `unshare()` system call to create new namespaces when spawning container processes[6].

3. System Architecture

3.1 Component Overview

The system consists of five interconnected subsystems:

1. Container Lifecycle Manager (control plane)
2. Memory and Storage Virtualization Layer
3. Resource Scheduler and Allocator
4. Security and Synchronization Module
5. Monitoring and Metrics System

3.2 Control Plane Architecture

Container Lifecycle Management

The control plane orchestrates container operations following the controller/worker model[8].

Core Components:

- API Server: Receives and processes container management requests

- Container Runtime: Creates and manages container processes
- State Store: Maintains current state of all containers
- Scheduler Interface: Communicates with resource scheduler

Container States:

Containers transition through states: pending → creating → running → stopping → stopped → terminated.

3.3 Worker Node Architecture

Container Execution Environment

Worker components execute on each node running containers[8].

- Container Agent: Receives instructions from control plane
- Process Monitor: Tracks container process health
- Resource Reporter: Sends metrics to monitoring system
- Network Manager: Configures container networking

3.4 Data Flow

Container Creation Flow:

1. User submits container specification to API
2. Scheduler determines optimal node placement
3. Container agent receives deployment instruction
4. Agent creates namespaces and configures cgroups
5. Container process starts with resource limits applied
6. Monitoring begins tracking container metrics

4. Team Role Distribution

4.1 Architecture Lead

Responsibilities:

- Design overall system architecture
- Implement container lifecycle management APIs
- Create container creation and deletion mechanisms
- Handle process isolation using namespaces
- Define interfaces between subsystems

Technical Deliverables:

- Container creation API with namespace configuration
- Container deletion with cleanup routines
- Process spawning with fork/exec and namespace creation
- State management system for container tracking
- Architecture documentation with diagrams

Implementation Focus:

Use Linux system calls (`fork()`, `execve()`, `unshare()`, `clone()`) to create isolated process environments[6].

4.2 Memory & Storage Manager

Responsibilities:

- Implement virtual memory partitioning per container
- Create file system isolation mechanisms
- Enforce storage quotas and limits
- Manage overlay file systems
- Handle memory pressure and OOM scenarios

Technical Deliverables:

- Memory cgroup configuration and enforcement
- Mount namespace implementation
- File system quota management
- Virtual file system layer
- Storage allocation tracking

Implementation Focus:

Use cgroups v2 memory controller for limit enforcement[10]. Implement mount namespace isolation using `chroot()` and `pivot_root()`[12].

4.3 Resource Scheduler

Responsibilities:

- Implement CPU time allocation algorithms
- Design fair scheduling across containers
- Enforce CPU and memory resource limits
- Create multi-level scheduling policies
- Monitor resource utilization

Technical Deliverables:

- CPU scheduler using proportional share scheduling
- Resource limit enforcement via cgroups
- Fair queuing algorithm for CPU distribution
- Dynamic resource allocation based on demand
- Resource monitoring and reporting system

Implementation Focus:

Implement weighted CPU share allocation where shares act as priorities during contention[10]. Consider both hard limits (quotas) and soft limits (shares)[7].

4.4 Security & Synchronization

Responsibilities:

- Implement inter-container communication mechanisms
- Create mutex and semaphore primitives for shared resources
- Design deadlock prevention algorithms
- Enforce security policies and access control
- Isolate IPC resources between containers

Technical Deliverables:

- IPC namespace isolation implementation
- Shared memory with access control
- Semaphore-based synchronization primitives
- Deadlock detection and prevention system
- Security policy enforcement layer

Implementation Focus:

Use IPC namespaces for isolation[9]. Implement resource ordering and timeout mechanisms to prevent deadlocks.

4.5 Monitoring & Testing

Responsibilities:

- Create container metrics dashboard
- Implement performance monitoring tools
- Conduct load testing scenarios
- Benchmark resource allocation efficiency
- Document testing methodologies and results

Technical Deliverables:

- Real-time metrics collection system
- Dashboard displaying CPU, memory, I/O metrics
- Load testing framework with concurrent containers
- Performance benchmark suite
- Comprehensive testing documentation

Implementation Focus:

Monitor cgroup statistics (`/sys/fs/cgroup/`) for resource usage[10]. Create stress tests with varying container counts and resource demands.

5. Implementation Guidelines

5.1 Development Environment Setup

Required Tools:

- Linux kernel 5.x+ (for cgroups v2 support)
- GCC/Clang compiler for C/C++ implementation
- Python 3.x for orchestration logic and API
- Root/sudo access for namespace and cgroup operations

System Libraries:

- libcap for capability management
- libseccomp for syscall filtering
- libnl for network namespace configuration

5.2 API Design

Container Management Endpoints:

```
POST /containers/create
GET /containers/{id}
DELETE /containers/{id}
POST /containers/{id}/start
POST /containers/{id}/stop
GET /containers/{id}/logs
GET /containers/{id}/stats
```

Request/Response Format:

Use JSON for API communication with declarative configuration specifying desired container state[14].

5.3 Testing Strategy

Unit Testing:

- Test namespace creation and isolation
- Verify resource limit enforcement
- Validate deadlock prevention logic

Integration Testing:

- Multi-container communication tests
- Resource contention scenarios
- Container lifecycle transitions

Load Testing:

- Spawn 100+ concurrent containers
- Measure scheduling fairness
- Test memory pressure handling
- Benchmark CPU allocation accuracy

5.4 Performance Metrics

Key Performance Indicators:

Metric	Measurement	Target
Container Start Time	Time from API call to running state	< 500ms
CPU Allocation Accuracy	Actual vs configured CPU shares	± 5%
Memory Isolation	Zero unauthorized memory access	100% isolation
Scheduler Fairness	Variance in CPU time distribution	< 10% deviation
Deadlock Frequency	Deadlock occurrences per 1000 ops	Zero

Table 3: Performance targets for container orchestration system

6. Technical Challenges & Solutions

6.1 Process Isolation

Challenge: Complete PID namespace isolation requires proper `/proc` remounting[6].

Solution: After creating PID namespace, unmount existing `/proc` and mount new procfs to show only container processes.

6.2 Resource Overcommitment

Challenge: Multiple containers requesting more resources than physically available[13].

Solution: Implement reservation (guaranteed) vs limit (maximum) model. Use memory overcommitment with careful monitoring and OOM killer configuration.

6.3 Container Networking

Challenge: Containers need network isolation but also external connectivity.

Solution: Create network namespaces with virtual ethernet pairs (veth). Implement bridge networking or overlay networks for inter-container communication.

6.4 Security Boundaries

Challenge: Root user inside container could escape isolation[9].

Solution: Use user namespaces to map container root to unprivileged host user. Apply seccomp filters to restrict dangerous system calls.

7. Advanced Features (Optional Extensions)

7.1 Dynamic Scaling

Implement autoscaling that creates/destroys containers based on resource utilization[3].

7.2 Container Migration

Enable live migration of containers between physical nodes for load balancing.

7.3 Persistent Storage

Implement volume mounting for persistent data across container restarts.

7.4 Network Policies

Create firewall rules controlling inter-container communication.

8. Evaluation Criteria

8.1 Functionality Assessment

- Container creation and lifecycle management working
- Resource isolation effective (CPU, memory, storage)
- Scheduling algorithm implements fairness
- IPC mechanisms functional
- Deadlock prevention operational

8.2 Performance Evaluation

- Container startup latency measurements
- Resource allocation accuracy validation
- Scheduling fairness quantification
- Scalability testing (container count limits)

8.3 Code Quality

- Proper error handling throughout
- Memory leak prevention
- Clean architecture with separation of concerns
- Comprehensive documentation

9. Project Timeline

Recommended 8-Week Schedule:

Week	Milestones
1-2	Architecture design, API specification, environment setup
3-4	Core implementation: process creation, namespaces, basic isolation
5-6	Resource scheduling, memory management, file system isolation
7	IPC mechanisms, security, deadlock prevention
8	Testing, optimization, documentation, presentation preparation

Table 4: Suggested project timeline with milestones

10. Learning Resources

10.1 Key Concepts

- Linux kernel namespaces and cgroups documentation
- Container orchestration fundamentals[1][2]
- Process scheduling algorithms
- Virtual memory management
- Distributed system design patterns

10.2 Reference Implementations

- Docker container runtime
- Kubernetes orchestration platform[2]
- LXC Linux containers
- NSJail lightweight isolation tool[12]

10.3 Testing Tools

- stress-ng for resource stress testing
- perf for performance profiling
- systemd-cgtop for cgroup monitoring
- bpftrace for kernel-level tracing

11. Conclusion

This container orchestration project provides comprehensive hands-on experience with fundamental operating system concepts. By implementing process isolation, memory virtualization, resource scheduling, and inter-process communication, team members gain deep understanding of how modern containerization technologies work at the kernel level.

The distributed role structure ensures each team member develops expertise in specific domains while contributing to a cohesive system. Success requires coordination, clear interfaces between components, and rigorous testing to verify isolation and resource management guarantees.

The resulting system demonstrates practical application of OS theory to solve real-world problems of resource management and workload

isolation in multi-tenant environments.

References

[1] Cisco. (2024, August 8). What Is Container Orchestration? https://www.cisco.com/site/us/en/learn/topics/computing/what-is-container-orchestration.html

[2] Red Hat. (2025, March 20). What is container orchestration? https://www.redhat.com/en/topics/containers/what-is-container-orchestration

[3] IBM. (2024, October 21). What Is Container Orchestration? https://www.ibm.com/think/topics/container-orchestration

[4] LogicMonitor. (2025, November 5). Container orchestration: Automate, scale, and optimize your apps. https://www.logicmonitor.com/blog/what-is-container-orchestration

[5] LaunchDarkly. (2026, February 22). What Is Container Orchestration, Exactly? Everything to Know. https://launchdarkly.com/blog/what-is-container-orchestration-exactly-everything/

[6] Dev.to. (2025, October 5). Understanding Linux Namespaces: A Guide to Process Isolation. https://dev.to/ajinkya_singh_2c02bd40423/understanding-linux-namespaces-a-guide-to-process-isolation-4gbg

[7] Docker. (2023, June 12). Efficient Strategies and Best Practices to Minimize Resource Consumption of Containers. https://dev.to/docker/efficient-strategies-and-best-practices-to-minimize-resource-consumption-of-containers-in-host-systems-2o5

[8] LaunchDarkly. (2026, February 22). What Is Container Orchestration? https://launchdarkly.com/blog/what-is-container-orchestration-exactly-everything/

[9] UDS Enterprise. (2024, July 21). Isolating systems with Linux Namespaces. https://udsenterprise.com/en/isolating-systems-linux-namespaces/

[10] AWS. (2019, October 20). How Amazon ECS manages CPU and memory resources. https://aws.amazon.com/blogs/containers/how-a

mazon-ecs-manages-cpu-and-memory-resources/

[11] Portworx. (2025, October 30). How Does Container Orchestration Work. https://portworx.com/knowledge-hub/how-does-container-orchestration-work/

[12] GitHub. (2015, May 13). google/nsjail. https://github.com/google/nsjail

[13] Tencent Cloud. (2025, July 13). What are the resource allocation strategies in container asset management. https://www.tencentcloud.com/techpedia/117614

[14] Mirantis. (2025, November 2). What Is Container Orchestration? https://www.mirantis.com/blog/container-orchestration/